nebio: CPU Measurement and Hardware Test

QB Nebulae V2 (RPi4) · nebio firmware (**feature_newHardWareHandling**) compared with the original firmware (**0419df1**) · measured 2026-10-16 on the device

1. Summary

nebio (libnebio.so) builds and runs on the Nebulae, and the hardware test passed: all inputs register, every LED works, and audio plays without underruns. The CPU measurement, however, shows that **the nebio firmware currently uses more CPU than the original firmware**. Both were measured with the SuperCollider bank, the same jackd settings, and nobody touching the module for 5 minutes:

	Original (0419df1)	nebio	Difference
Whole Pi busy (4 cores = 100 %)	26.5 %	46.0 %	**+19.5 points (about +0.8 core)**
python3 process (% of one core)	81.6 %	101.7 %	+20.1
Python main loop	74.2 %	79.0 %	+4.8 (about the same)
nebio input thread	-	20.2 %	+20.2 (new)
Kernel time without a process (% of Pi)	2.7 %	16.8 %	+14.1
I2C interrupts/s (LED writes)	4 680	986	**-79 %**
SPI interrupts/s (ADC reads)	5 266	7 996	+52 %
python3 memory (max RSS)	307 MB	80 MB	**-227 MB**

- **The Python main loop costs about the same in both firmwares** (74 % vs 79 % of a core). Moving the hardware access into C++ did not make the Python side cheaper, because updateAll() and the control logic dominate, not the I/O calls.
- **nebio adds two costs on top:** its 1 kHz input thread (20 % of a core) and about 14 points of extra kernel time that is not charged to any process.
- **nebio is better at LEDs and memory:** almost 5 times fewer I2C interrupts, and the Python process uses 227 MB less memory.
- SuperCollider and JACK are the same in both (sclang 6-7 %, scsynth 3.5 %, jackd 4 %).

Section 7 lists what to change so that nebio ends up cheaper than the original. The biggest single item is the extra kernel time, whose cause is not proven yet.

Status

Item	Result
libgpiod installed on the Pi	Done: libgpiod 2.3.1 (cmake, gcc, make, pkgconf, liblo 0.34 were already there)
nebio build on the Pi	Done: libnebio.so, 0 warnings (armv7h, gcc 15.2)
Hardware test firmware (hwtest.py)	Runs at boot, log on the USB stick
Inputs: knobs, CV, buttons, gates, encoders	All register (user test)
LEDs: 5 button LEDs, 4 RGB LEDs x 3 colours	All work (ledwalk.py, checked by eye)
Audio during the test	Plays, 0 underruns
Normal nebulae with nebio	Starts, SC bank plays
CPU: SC bank, both firmwares	Done (this report)
CPU: Csound and PD banks	Not measured yet

2. Setup

Item	Value
Device	Raspberry Pi 4, 4 cores, Arch Linux ARM **armv7h** (32 bit), kernel 5.15.43-2-rpi-ARCH

Item	Value
CPU clock	governor performance, 1200 MHz (both runs)
nebio firmware	feature_newHardWareHandling, libnebio.so built 2026-10-16 07:32, nebulae.py md5 e67c4c6f8289. Measured 08:41-08:46 (Pi clock)
Original firmware	commit 0419df1 (the commit before nebio, python3, RPi.GPIO, Adafruit drivers, python-osc), nebulae.py md5 64995ce48aba, checked against git. Measured 10:20-10:25
Running	nebulae.py (python3), sclang, scsynth, jackd (-p128 -n3 -r48000, hw:sndrpiproto)
Instrument	SC bank, the instrument loaded at boot
Conditions	No user input, temperature 44-47 °C in both runs, never throttled
Window	300 s each, one sample per second
Tool	Code/scripts/cpustat.sh (sh + awk + /proc only)

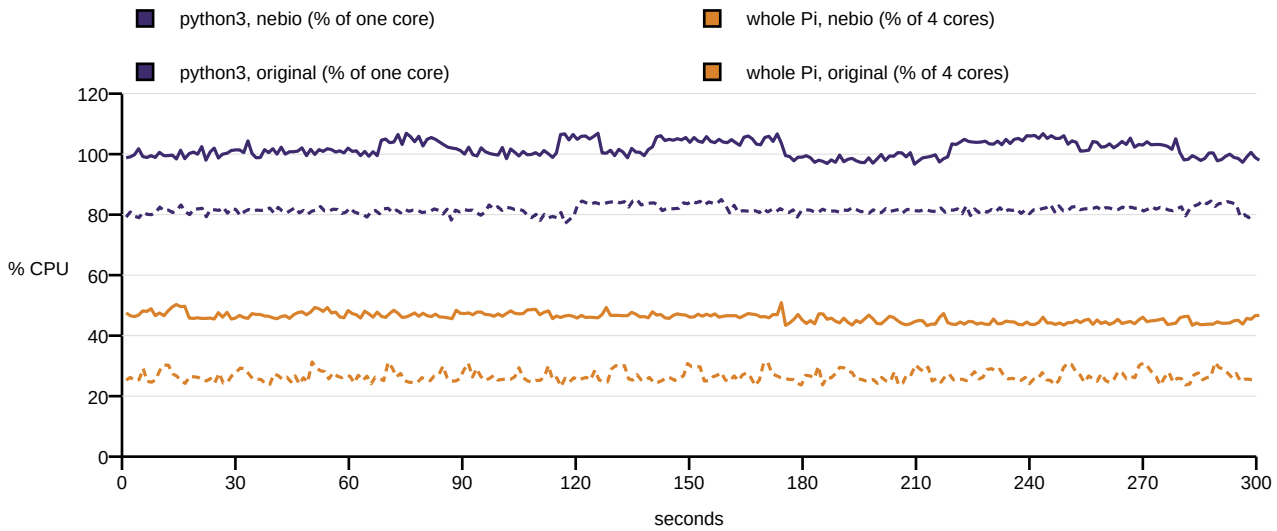
CPU values are given as **% of one core**, the same as top (100 % = one full core). "Share of Pi" is the same value divided by 4. The whole-Pi line uses /proc/stat, where all 4 cores = 100 %.

3. Results side by side: SC bank

Program	Firmware	min %	mean %	p95 %	max %	share of Pi	max RSS
python3	original	77.1	81.6	84.0	85.0	20.4 %	307 MB
	nebio	96.7	**101.7**	106.0	106.9	25.4 %	80 MB
sclang	original	4.5	5.7	6.3	7.2	1.4 %	29 MB
	nebio	5.4	6.9	7.3	8.9	1.7 %	29 MB
scsynth	original	2.7	3.6	4.5	4.5	0.9 %	120 MB
	nebio	1.8	3.5	4.5	4.6	0.9 %	118 MB
jackd	original	3.6	4.2	5.4	5.4	1.1 %	127 MB
	nebio	2.7	4.5	5.5	5.5	1.1 %	127 MB
whole Pi	original	23.2	26.5	30.1	31.3		
(4 cores = 100 %)	nebio	43.3	**46.0**	48.4	50.9		
temperature °C	original	43.8	45.2	46.3	46.7		
	nebio	43.8	45.3	46.3	46.7		

pd and csound were not running in either measurement (Csound runs inside python3 in the Csound bank).

CPU over the 5 minutes



Solid lines: nebio. Dashed lines: original. Both are flat: no spikes, nothing grows over time.

Inside the python3 process

Thread	Original	nebio	What it is
main thread	74.2 % (max 78.0)	**79.0 %** (max 84.2)	Python main loop: updateAll(), ControlHandler, ui.py
nebio input thread	-	**20.2 %** (max 21.9)	1 kHz scan of shift register, encoders, ADCs
other nebio threads	-	2.4 %	LED writer, gate events, reset LED PWM
Python threads (Thread-2, Thread-3)	6.3 %	-	Python worker threads of the original firmware
other	1.1 %	-	

The nebio threads are not named yet, so they show as python3; the input thread is identified by its steady 20 %, which matches the hardware test.

4. Kernel work and interrupts

This kernel uses tick-based CPU accounting without IRQ time accounting (CONFIG_HZ=100). Interrupt work that happens while a core is idle is counted as system time without any process, so it is invisible in top. The difference between the whole-Pi load and the measured programs shows it:

	Original	nebio
user / system / idle (% of Pi)	18.8 / 7.5 / 73.4	26.5 / 19.3 / 54.0
Measured programs (% of Pi)	23.8 %	29.2 %
Kernel time without a process	**2.7 %**	**16.8 %**

Interrupt source (per second)	Original	nebio	Cause
IPI3 function call	14 973	9 582	Cores signalling each other
fe204000.spi	5 266	**7 996**	ADC reads. nebio: 2 chips x 8 channels x 500 Hz, one interrupt per channel transfer
fe804000.i2c	**4 680**	986	LED writes to the PCA9685
arch_timer	2 718	1 289	Kernel timer
DMA IRQ x 2	374 + 374	375 + 375	Audio stream (48000 / 128). Same
IPI2 rescheduling	22	29	

The interrupt counts alone do not explain the extra kernel time: nebio has **fewer** interrupts in total (about 21 000/s against 28 000/s), yet about 14 points more unaccounted system time. Short-lived processes were ruled out (2 new processes in 10 s). The most likely candidates are the per-transfer cost of the SPI path (each ADC channel is its own transfer, 8000 per second) and the 1 kHz wake-ups of the input thread, but this is **not proven yet**. The next step is to measure it directly: run nebio with the ADC rate lowered, then with the scan rate lowered, and see which one removes the extra system time.

5. Hardware test results (nebio)

hwtest.py ran at boot instead of nebulae (firmware built with `create_fw_update.sh --hwtest`), with audio playing and the log written to the USB stick.

- **Inputs:** all knobs, both encoders, buttons and gates registered (118 digital events). CV 4 (pitch) and CV 1 (speed) were seen moving; the other CV inputs were not patched in that run.
- **nebio:** scan 1000 Hz, worst scan 1.0-2.4 ms, test process about 22 % of one core, 41-42 °C, not throttled.
- **Audio:** 12 files looped on plughw:sndrpiproto, **0 underruns**.
- **LEDs:** the PCA9685 registers were read while the test ran and matched the expected values. ledwalk.py then lit every LED and colour on its own; all worked. In hwtest.py the pitch/speed LEDs do not behave like nebulae (encoders only dim the _neg LEDs, knobs set the _pos colours), which looked like "not reacting".

6. Other findings

- **sudo** on the Pi asks for the root password, not alarm's.
- **pacman 7** needs `--disable-sandbox` on kernel 5.15 (no Landlock). The package database was stale (mirror 404); after `-Sy` glibc matched the repo, so libgpiod was installed on its own without a partial upgrade.
- **The Pi is 32 bit (armv7h)**, not aarch64 as the nebio step 1 report and the SC PKGBUILD assume.
- **The nebio build folder is owned by root** (built at boot by startup.sh), so manual builds must run as root.
- **Older python2 firmwares do not run on this image**: Python 2 has no OSC module (pyOSC) and a broken hashlib. A firmware from the niklassandberg branch crashed in a restart loop with `ImportError: No module named OSC`. OSC.py was copied into the Python 2 site-packages from /home/alarm/usr/lib/python2.7/site-packages; it is not needed by the python3 firmwares.
- **USB stick dropped out during one boot** (sda "device offline or changed", I/O errors). Copying an .instr and a .pd file from the stick failed. Not related to nebio.
- **Code/log is missing early in boot**, so the first neb_log.txt lines are lost. Harmless.

7. What to change (not done)

Goal: the nebio firmware should use less CPU than the original, not more.

Change	Expected effect
Find the source of the extra kernel time (ADC rate and scan rate experiments, section 4)	Up to 14 points of the Pi, the largest part of the difference
ADC reads at 250 Hz instead of 500 Hz	Halves the SPI transfers (8000/s), below the original's 5266/s
Read the input thread at a lower rate, or wait for edges instead of polling	Reduces the 20 % input thread
Pace the Python main loop lower (for example 200 Hz instead of about 500 Hz)	The 79 % main thread scales with the loop rate. Knob smoothing filters work per loop and would need retuning.
Name the nebio threads (pthread_setname_np)	top and cpustat.sh show nebio-input, nebio-leds, ... instead of python3

8. Repeating the measurement

Same instrument, nobody touching the module, script copied to /tmp:

```
sudo sh /tmp/cpustat.sh -d 300 -l "<bank>, <firmware>"
# -> /tmp/cpustat_<date>.txt (report) and .csv (one row per second)
# /tmp is cleared at reboot: copy both files off before rebooting
```

Still to do: the Csound and Pure Data banks on both firmwares (in the Csound bank the python line includes Csound), and a new nebio run after each change in section 7.

Files

File	Content
measurements/cpustat/nebio_sc_20261016_084140.txt / .csv	nebio firmware, SC bank
measurements/cpustat/orig_sc_20261016_102031.txt / .csv	original firmware 0419df1, SC bank
measurements/cpustat/make_report.py	Builds this PDF
Code/scripts/cpustat.sh	Measurement script
Code/nebio/tools/hwtest.py, ledwalk.py	Hardware test and LED walk